



C++: Exception Handling

IF2210 – Semester II 2025/2026

Tujuan

- Di akhir sesi, peserta diharapkan mampu untuk:
 - Mendefinisikan *exception*
 - Menangani *exception* dengan menggunakan blok try-catch
 - Membuat kelas yang dilengkapi dengan class Exception sendiri

Penanganan Kesalahan pada Program

Penambahan kode khusus untuk menangani kesalahan dalam eksekusi program membuat program menjadi rumit

```
// di definisi fungsi:
int hitung() {
    // ...
    if (...) // periksa kondisi
                // kesalahan
        return NILAI_KHUSUS;
    else
        return NILAI_FUNGSI;
}
```

```
// di penggunaan fungsi:
if (hitung() == NILAI_KHUSUS) {
    // ... jalankan instruksi
    // untuk menangani kesalahan
} else {
    // ... lanjutkan instruksi
    // untuk kasus normal
}
```

Exception

- Exception adalah mekanisme untuk menghentikan eksekusi normal suatu operasi ketika kontraknya tidak dapat dipenuhi.
- Penanganan *exception* pada C++ melibatkan **throw**, **catch**, dan **try**
- **return** = kembali dari fungsi secara normal;
- **throw** = operasi gagal secara konseptual; serahkan kendali ke pemanggil yang siap menangani kondisi tersebut.

```
int hitung() {  
    // ...  
    if (...) // periksa kondisi  
           // kesalahan  
        throw "Ada kesalahan";  
    else  
        return NILAI_FUNGSI;  
}
```

Exception

- *Error* tidak harus ditangani dengan *exception handling* (namun *exception* mempermudah penanganan *error*).
- *Exception* bekerja dengan cara mengubah alur eksekusi program sambil melempar suatu objek tertentu sebagai informasi untuk alur yang baru.
- Sebuah *event* yang akan menginterupsi alur proses program normal dari sebuah program.
- Pastikan perilaku *exception* dikomunikasikan pada kontrak suatu fungsi / method.

Membuat Exception

- Objek yang diciptakan disebut *exception object*
- *Exception object* mengandung informasi tentang *error* tersebut (termasuk tipe dan state program ketika *error* terjadi)
- Menciptakan *exception object* dan melempar ke *runtime system* disebut *throwing an exception* (keyword `throw` di C++)
- Setelah *method* melempar *exception*, *runtime system* akan mencari sesuatu untuk *meng-handle* itu = disebut *exception handler*
- *Exception handler* akan melakukan *catch the exception*
 - Jika tidak di-*handle* (di-*catch*) akan mengakibatkan program *terminate abnormally*

Exception dan Invariant

- Ketika suatu method melempar exception:
 - Objek yang sedang digunakan harus tetap berada dalam keadaan valid.
 - Invariant kelas tidak boleh dilanggar.
- Oleh karena itu:
 - Perubahan state sebaiknya dilakukan setelah operasi yang berpotensi gagal berhasil.
 - Resource harus dikelola dengan RAI agar tetap aman jika terjadi exception.

Exception Handling

- **try block** (keyword `try` di C++)
 - Berisi kode yang mungkin memunculkan *exception*
 - *Try block* bisa dibuat untuk setiap kode yang mungkin menimbulkan *exception*
 - Bisa juga dengan mengumpulkan banyak kode dalam sebuah *try block*
- **catch block** (keyword `catch` di C++)
 - Berisi kode yang merupakan *exception handler*
 - Menangani *exception* dengan tipe yang sesuai dengan tipe yang ditunjukkan pada argumen
- Jika sebuah *method* ditulis menangani *exception*, sebaiknya dilakukan invokasi dalam sebuah blok **try ... catch**

Exception Handling

catch dituliskan setelah blok **try** untuk menangkap exception yang di-**throw**

```
try {  
    // ...  
    int hasil = hitung();  
    // ...  
}  
catch (std::exception& e) {  
    // ... jalankan instruksi untuk menangani kesalahan  
}  
  
// eksekusi berlanjut pada bagian ini...
```

Stack Unwinding dan RAII

- Saat alur program berubah akibat terjadinya exception, objek-objek yang telah dibuat akan dihancurkan secara otomatis sesuai urutan stack (stack unwinding).
- Ketika exception dilempar:
 - Stack akan di-unwind.
 - Destructor dari objek lokal akan dipanggil secara otomatis.
- Karena itu:
 - Resource harus dibungkus dalam objek (RAII),
 - Hindari raw pointer yang memerlukan delete manual.

Exception Handling

- Perintah dalam blok **catch** = *exception handler*
- Sebuah blok try dapat memiliki > 1 *exception handler*, masing-masing untuk menangani tipe *exception* yang berbeda
- Tipe *handler* ditentukan “*signature*”-nya. catch-all handler memiliki “signature” ...
- Penggunaan catch-all sebaiknya terbatas, karena dapat menyembunyikan tipe *exception* yang lebih spesifik dan menyulitkan reasoning desain.

Sintaks Catch-All

```
try {  
    // deretan instruksi  
}  
catch (StackEmpty&) {  
    // handler untuk tipe kesalahan StackEmpty  
}  
catch (StackFull&) {  
    // handler untuk tipe kesalahan StackFull  
}  
catch (...) { // literally pakai tiga tanda titik  
    // handler untuk semua jenis exception lainnya  
}
```

Pemilihan Handler

- Jika terjadi *exception*, hanya satu *handler* yang dipilih berdasarkan “*signature*”-nya
- Instruksi di dalam *handler* yang terpilih dijalankan
- Eksekusi dari blok **try** tidak dilanjutkan
- Eksekusi berlanjut ke bagian yang dituliskan setelah bagian **try-catch** tersebut.

Contoh Kelas untuk Stack Exception

```
#include <stdexcept>

class StackEmpty : public std::runtime_error {
public:
    StackEmpty()
        : std::runtime_error("Stack is empty") {}
};

class StackFull : public std::runtime_error {
public:
    StackFull()
        : std::runtime_error("Stack is full") {}
};
```

Melemparkan dan Menangani Exception

Di implementasi kelas Stack:

```
int Stack::pop() {  
    if (empty()) {  
        throw StackEmpty();  
    } else {  
        int v = data_.back();  
        data_.pop_back();  
        return v;  
    }  
}
```

Di kode yang menggunakan Stack:

```
Stack s;  
// ...  
try {  
    int x = s.pop();  
}  
catch (const StackEmpty& e) {  
    std::cerr << e.what()  
                << std::endl;  
}
```

Exception sebagai Informasi, Bukan Kontrol Alur

- Exception digunakan untuk kondisi yang luar biasa (exceptional), bukan sebagai mekanisme kontrol alur normal.
- Gunakan return value untuk kondisi yang diharapkan, gunakan exception untuk kegagalan kontrak.

Chain Exception

- Method bisa merespon terjadinya exception dengan melempar exception lagi

```
try {  
    // ...  
}  
catch (IOException& e) {  
    throw SampleException("Another Exception", e);  
}
```

- Saat sebuah method menangkap exception, pikirkan tanggung jawab method tersebut untuk memutuskan:
 - Tangani sendiri, atau
 - Lempar exception lagi?

CPP Standard Exception

```
// Example: bad_alloc standard
//           exception, i.e.,
//           memory full
#include <iostream>
#include <exception>
using namespace std;
int main() {
    try {
        int* myarray = new int[1000];
    } catch (exception& e) {
        cout << "Standard exception: " << e.what() << endl;
    }
    return 0;
}
```

exception	description
bad_alloc	thrown by new on allocation failure
bad_cast	thrown by dynamic_cast when it fails in a dynamic cast
bad_exception	thrown by certain dynamic exception specifiers
bad_typeid	thrown by typeid
bad_function_call	thrown by empty function objects
bad_weak_ptr	thrown by shared_ptr when passed a bad weak_ptr

exception	description
logic_error	error related to the internal logic of the program
runtime_error	error detected during runtime

Takeaway

- Exception digunakan untuk menyatakan bahwa suatu operasi tidak dapat memenuhi kontraknya.
- Dengan exception:
 - Alur normal dipisahkan dari penanganan kegagalan.
 - Invariant objek dapat dijaga dengan lebih jelas.
 - Resource tetap aman melalui mekanisme RAI dan stack unwinding.
 - Penanganan kegagalan dapat dilakukan pada level yang paling tepat.
- Exception bukan mekanisme kontrol alur biasa, melainkan alat untuk menjaga konsistensi dan keandalan sistem.